



中山大學
SUN YAT-SEN UNIVERSITY

表达式与运算符

中山大学计算机学院



讲课人：课题组

目录

CONTENTS

01

算术运算符

02

逻辑运算符

03

关系运算符

04

位运算符

05

赋值运算符

06

其他运算符

07

运算符优先级



中山大學
SUN YAT-SEN UNIVERSITY

标识符与关键字

Identifiers and Keywords



标识符 (Identifiers)

- 标识符是**数字**、**下划线**、小写及大写拉丁**字母**和以 `\u` 及 `\U` 转义记号指定的 **Unicode 字符 (C99 起)** 的任意长度序列。
- 合法的标识符必须以**非数字字符** (拉丁字母、下划线或 Unicode 非数字字符 (C99 起)) **开始**。
- 标识符**大小写有别** (小写和大写字母不同) 。

```
#include <stdio.h>

int main() {
    char* cat = "cat";
    char* Cat = "cat"; ✓规则3, cat与Cat是不同标识符
    char* \U0001f431 = "cat"; // Since C99
    char* 1Cat = "cat"; ✗规则2, 必须非数字开始
    printf("%s", \U0001f431);
}
```



标识符的作用

- 标识符能指代下列类型的实体**名称**
 - 对象（变量）
 - 函数
 - 标签（struct、union 或枚举）
 - 结构体或联合体成员
 - 枚举常量
 - typedef 名
 - 标号名
 - 宏名
 - 宏形参名



使用有意义，可读的标识符

- 以下哪些标识符是合法的

age_of_dog ✓
Age#

taxRateY2K ✓
Age-Of-Cat

PrintHeading ✓
ageOfHorse

_EAX ✓
2000TaxRate

- 以下哪些是好的标识符

PRINTTOPPORTION
Printtopportion
Print_Top_Portion
pRiNtToPpOrTion

✓
PRINT_TOP_PORTION
printTopPortion
PrintTopPortion
isPrintTopPortion



常见的标识符命名风格 (Style)

- **Unix like 风格**: 单词用大写或小写字母, 单词间用下划线风格
 - 大写风格: 常用于常量
 - 例如: PI, E, INT_MAX
 - 小写风格: 常用于局部的变量、函数等
 - 例如: printf, i, x, length, my_age, text_color
- **驼峰 (camelCase) 风格**: 首字母大写的单词连接一起形成驼峰一样
 - 大驼峰风格: 常用于全局事物如类型名、结构名、函数等
 - 例如: Point, Vector, Color, WindowStyle, ContextMenu
 - 小驼峰风格: 常用于局部的变量、函数等
 - 例如: myAge, textColor, firstName, numberOfYears, annualInterestRate
- **匈牙利风格**: 用一个前缀表示数据类型。
 - 例如, 用is作为前缀表示布尔数据。isOK;



C 关键词 (keywords)

- C 中保留的标识符 (Reserved identifiers) 列表。因为语言使用这些关键词，故不可重定义它们。

auto	float	signed	_Alignas (C11 起)
break	for	sizeof	_Alignof (C11 起)
case	goto	static	_Atomic (C11 起)
char	if	struct	_Bool (C99 起)
const	inline (C99 起)	switch	_Complex (C99 起)
continue	int	typedef	_Decimal128 (C23 起)
default	long	union	_Decimal32 (C23 起)
do	register	unsigned	_Decimal64 (C23 起)
double	restrict (C99 起)	void	_Generic (C11 起)
else	return	volatile	_Imaginary (C99 起)
enum	short	while	_Noreturn (C11 起)
extern			_Static_assert (C11 起)
			_Thread_local (C11 起)

随着C语言的发展，关键字越来越多，详细参考：

[C 关键词 - cppreference.com](https://en.cppreference.com/w/cpp/keyword)

[标识符 - cppreference.com](https://en.cppreference.com/w/cpp/keyword)



C 关键词的记忆

- C 基本数据类型名称
 - char, short, int, long, float, double, void
 - 建议不要使用的标识符：已下划线开头的首字母大写单词，如 `_Bool`；以 `_t` 结尾，如 `size_t`
- 对象的修饰、限定或属性名称
 - signed, unsigned, long, const, static, extern, volatile, auto, register
- 复杂数据申明
 - struct (结构体申明) , union (共用体申明) , enum (枚举申明) , typedef (类型别名申明)
- 特殊操作
 - sizeof
- 程序结构控制
 - if, else, switch, case, default, for, while, do, return, continue, break, goto



中山大學
SUN YAT-SEN UNIVERSITY

表达式

國立中山大學

Expressions



表达式 (Expressions)

- 表达式是运算符 (Operators) 及其操作数 (Operands) 的序列，它指定一个运算。例如：

2+3

- 其中，“+”是运算符，“2”和“3”是操作数
- 其结果 (Result) 是“5”
- 常见操作符 (右表)

Common operators						
assignment	increment decrement	arithmetic	logical	comparison	member access	other
<pre>a = b a += b a -= b a *= b a /= b a %= b a &= b a = b a ^= b a <<= b a >>= b</pre>	<pre>++a --a a++ a--</pre>	<pre>+a -a a + b a - b a * b a / b a % b ~a a & b a b a ^ b a << b a >> b</pre>	<pre>!a a && b a b</pre>	<pre>a == b a != b a < b a > b a <= b a >= b</pre>	<pre>a[b] *a &a a->b a.b</pre>	<pre>a(...) a, b (type) a ?: sizeof _Alignof (since C11)</pre>

表达式的七个类别



初等表达式与子表达式

- 表达式

$$1 + 2 * 3$$

- “+” 是运算符，操作数 “1” 是**初等表达式** (Primary expressions) ，操作数 “2 * 3” 是**子表达式** (Subexpressions)
- 初等表达式不能在分解，可能是以下几种
 - 1) 常量及字面量 (例如 2 或 "Hello, world")
 - 2) 适合的已声明标识符 (例如 n 或 printf)
 - 3) 泛型选择(C11 起)



表达式学习要点

- 右边程序是经典的选择题，问哪条赋值语句是错误的？
 - 为什么第三句是错误的？
- C 表达式学习的要点：
 1. 类型 (object type)
 2. 值类别 (value category)
 3. 求值顺序 (结合性)
 4. 运算符优先级 (precedence)
 5. 兼容类型与隐式转换

```
#include <stdio.h>

int main() {
    int i,j;
    i = j = 1; //赋值运算先算右表达式
    i = (j = 1); //赋值运算返回左值
    (i = j) = 1; //括号改变了计算的顺序
                //编译提示: (i = j)不是左值表达式
}
```



值类别 – 左值、右值

- C 中每个表达式有两个独立的属性
 - 左值表达式求值得到的**数据类型**。例如：7L 是长整形表达式
 - 表达式属于三个**值类别**之一：**左值**、非左值对象（**右值**）以及函数指代器。
- 左值表达式
 - 左值表达式**可作为**赋值运算符的左运算数。如变量
 - 左值表达式求值得到对象标识（或一个内存变量）。如果表达式是左值，则
 - 可以取地址
 - 使用自增运算符
- 右值表达式（非左值对象）
 - 右值表达式**只能作为**赋值运算符的右运算数。如常量
 - 表达式计算结果只是一个数值，没有存储位置



中山大學
SUN YAT-SEN UNIVERSITY

运算符

學大山中立國
Operators



从运算到运算符

运算在数学上是一种行为，是通过已知量的可能的组合获得新的量。运算的本质是集合之间的映射。而运算符就是反映运算类型的符号。

例如，算术中的加法 $5 + 3 = 8$ ，这里 5 和 3 是**操作数 (operands)**，8 是**结果 (result)**，而加号 “+” 是**运算符 (operator)** 表明这是一个加法运算。这是一个常见的**二元运算**，本质上是 $A \times B \rightarrow C$ 形式的映射。

C 语言内置了丰富的运算符，大致将其分为以下六类：算术运算符、赋值运算符、逻辑运算符、关系运算符、位运算符和其他运算符。

备注：不同教材运算符分类可能不同



运算符 – 优先级与结合性

```
/*operator precedence*/  
#include <stdio.h>  
int main() {
```

```
    int a = 16, b = 4, c = 2;
```

```
    int d = a + b * c;
```

```
    int e = a / b * c;
```

```
    printf( "d=%d, e=%d\n", d, e);
```

```
    return 0;
```

```
}
```

乘性运算优先加减运算

二元算术运算
左结合



运算符-优先级与结合性

```
/*operator eval order*/  
#include <stdio.h>  
  
int main() {  
  
    int a=4,b=2,c=3;  
  
    printf("%d\n",a=b=c);  
    printf("%d\n", a=b==c);  
    printf("%d\n",a==(b=c));  
    printf("%d\n", a==(b==c));  
  
    return 0;  
}
```



运算符-优先级与结合性

优先级，就是当多个运算符出现在同一个表达式中时，先执行哪个运算符。

结合性，就是当一个表达式中出现多个优先级相同的运算符时，先执行哪个运算符：先执行左边的叫左结合性，先执行右边的叫右结合性。

类别	运算符	结合性
后缀	() [] -> . ++ --	从左到右
一元	+ - ! ~ ++ -- (type)* & sizeof	从右到左
乘除	* / %	从左到右
加减	+ -	从左到右
移位	<< >>	从左到右
关系	< <= > >=	从左到右
相等	== !=	从左到右
位与 AND	&	从左到右
位异或 XOR	^	从左到右
位或 OR		从左到右
逻辑与 AND	&&	从左到右
逻辑或 OR		从左到右
条件	?:	从右到左
赋值	= += -= *= /= %=> >= <<= &= ^= =	从右到左
逗号	,	从左到右

算术运算 (Arithmetic operators)

算术运算符应用标准数学运算符到其操作数。

- 操作数类型
 - 字符, 整数, 实数, 复数, 布尔 (C99)
 - [算术类型 - cppreference.com](http://cppreference.com)
- 值类别
 - 非左值, 即
 - 不能放在赋值语句左边, 不能求地址等
- 求值顺序
 - 从左到右结合性 (非一元操作), 即
 - $a + b + c$ 被分析为 $(a + b) + c$
- 优先级: ...
- 隐式转换
 - 两个操作数都转为相同的类型
 - [隐式转换 - cppreference.com](http://cppreference.com)

运算符	运算符名	示例	结果
<code>+</code>	一元加	<code>+a</code>	a 提升后的值
<code>-</code>	一元减	<code>-a</code>	a 的相反数
<code>+</code>	加法	<code>a + b</code>	a 加 b
<code>-</code>	减法	<code>a - b</code>	从 a 减去 b
<code>*</code>	乘法	<code>a * b</code>	a 和 b 的积
<code>/</code>	除法	<code>a / b</code>	a 除以 b 的商
<code>%</code>	模	<code>a % b</code>	a 除以 b 的余数
<code>~</code>	逐位非	<code>~a</code>	a 的逐位非
<code>&</code>	逐位与	<code>a & b</code>	a 和 b 的逐位与
<code> </code>	逐位或	<code>a b</code>	a 和 b 的逐位或
<code>^</code>	逐位异或	<code>a ^ b</code>	a 和 b 的逐位异或
<code><<</code>	逐位左移	<code>a << b</code>	a 左移 b 位
<code>>></code>	逐位右移	<code>a >> b</code>	a 右移 b 位

算术运算

位运算



算术隐式转换规则-非整数

- 如果两个操作数类型不一样，则会发生隐式类型转换。
- 类型等级
 - `complex > imaginary > long double > double > float > 整数`
- 低等级操作数转换到高等级，例如：
 - `1.f + 2000000L; // long int 转 float; 执行 float 加法`
 - `1.0 + 1.0f + 1; //左结合, float 转 double, 执行double加法; int 转 double, 执行 double 加法`
- 应用：
 - 两个整数 `i, j` 需要执行浮点除法，表达式写法：
`1.0 * i / j` 或 `(double)i / j`



算术隐式转换 (Implicit Casting) 规则-整数

- 整数类型等级
 - long long > long >= int >= short > char
- 如果有高于 int 等级, 则低等级类型操作数转为高等级类型
- 否则操作数都自动转为对应的 int 或 unsigned int, 称为**整数提升**。
 - 提升后运算, unsigned 优先
- 例如:
 - a_char + 'a' //两个操作数转为int
 - 2 + 3LLU // int 转为 unsigned long long
 - 2u - 10 // 10 转为 unsigned int, 结果是4294967288 (即 UINT_MAX-7)
 - 2u - 10LL // 2 从 unsigned 转 long long, 结果是 -8

```
/*ops arithmetic int promotions*/  
#include <stdio.h>
```

```
int main() {  
    char i,j;  
    /*请分别输入 1, 2, 100, 129*/  
    scanf("%d",&i);  
    j = 255 * i;  
    printf("%d,%d,%0x\n",j,255 *i,  
            255 *i);  
}
```

```
-1,255,ff  
-2,510,1fe  
-100,25500,639c  
127,-32385,ffff817f
```



显式类型转换 (Explicit Casting)

- 从一种类型转变为另一种兼容类型。

语法

(类型名) 表达式

其中

类型名 - `void` 类型或任何标量类型

表达式 - 任何标量类型表达式 (除非 类型名是 `void` , 此情)

- 例如:
 - `(double)i`
 - `(long long)(i+j)`
- 隐式类型转换主要是方便我们书写习惯的算术表达式
- 当需要指定操作数类型时, 则可采用显式转换

```
C a.c > ...
1  #include <stdio.h>
2
3  int main()
4  {
5      int a = 21, b = 4;
6      double c,d;
7      c = (double) a / b;
8      d = (double)(a / b);
9      printf("c : %f\n d : %f\n", c ,d);
10
11 }
```

算术运算符 – 求商与取余

假设变量 A 的值为 10，变量 B 的值为 20，则：

运算符	描述	实例
+	把两个操作数相加	A + B 将得到 30
-	从第一个操作数中减去第二个操作数	A - B 将得到 -10
*	把两个操作数相乘	A * B 将得到 200
/	分子除以分母	B / A 将得到 2
%	取模运算符，整除后的余数	B % A 将得到 0

“/” 分为整数除法和浮点除法。例如：

$$25 / B = 1 \dots 5, 25.0 / B = 1.25$$

在C语言中，如果操作数是整数，表达式 25 / B 得到商 1；25 % B 得到余数 5

如果操作数是浮点数，表达式 25.0 / B 得到实数商 1.25；% 运算为非法



算术运算符- 案例1

```
/*ops arithmetic examples*/
#include <stdio.h>

int main() {
    int a = 20;
    int b = 10;
    int c ;
    c = a + b;
    printf("Line 1 - c 的值是 %d\n", c );
    c = a - b;
    printf("Line 2 - c 的值是 %d\n", c );
    c = a * b; printf("Line 3 - c 的值是 %d\n", c );
    c = a / b; printf("Line 4 - c 的值是 %d\n", c );
    c = a % b; printf("Line 5 - c 的值是 %d\n", c );
}
```

当左侧的代码被编译和执行时，它会产生下列结果：

Line 1 - c 的值是 30

Line 2 - c 的值是 10

Line 3 - c 的值是 200

Line 4 - c 的值是 2

Line 5 - c 的值是 0



算术运算符-案例2

```
/*ops arithmetic quotient*/
#include <stdio.h>

int main() {
    int a = 0123456;
    int i = 3;

    printf("input a octal:");
    scanf("%o",&a);

    /*calculate the i'th number*/
    int q = a >> (3 * (i-1)); //a / 8^(i-1)
    int r = q % 8;

    printf("The %i'th number of the octal %o is %d.",i,a,r);
}
```

位运算符

什么是位运算？位运算符作用于位，并逐位执行操作。 $\&$ 、 $|$ 和 $\wedge$ 的真值表如下所示：

p	q	$p \& q$	$p q$	$p \wedge q$
0	0	0	0	0
0	1	0	1	1
1	1	1	1	0
1	0	0	1	1



位运算-例子

A = 0011 1100

B = 0000 1101

A&B = 0000 1100

A|B = 0011 1101

A^B = 0011 0001

~A = 1100 0011

假设 A = 60, 且 B = 13, 现在以二进制格式表示, 它们如左所示。



位运算符

- 位运算的优先级：由高到低的顺序是： $\sim \rightarrow \ll, \gg \rightarrow \& \rightarrow | \rightarrow ^$;
- 位运算的运算对象只能是整型（int）或字符型（char）的数据；
- 位运算是对其运算量的每一个二进制位分别进行；



位运算符

假设变量 A 的值为 60，变量 B 的值为 13，则：

运算符	描述	实例
&	按位与操作，按二进制位进行"与"运算。运算规则： $0 \& 0 = 0$; $0 \& 1 = 0$; $1 \& 0 = 0$; $1 \& 1 = 1$;	(A & B) 将得到 12，即为 0000 1100
	按位或运算符，按二进制位进行"或"运算。运算规则： $0 0 = 0$; $0 1 = 1$; $1 0 = 1$; $1 1 = 1$;	(A B) 将得到 61，即为 0011 1101
^	异或运算符，按二进制位进行"异或"运算。运算规则： $0 \wedge 0 = 0$; $0 \wedge 1 = 1$; $1 \wedge 0 = 1$; $1 \wedge 1 = 0$;	(A ^ B) 将得到 49，即为 0011 0001
~	取反运算符，按二进制位进行"取反"运算。运算规则： $\sim 1 = -2$; $\sim 0 = -1$;	(~A) 将得到 -61，即为 1100 0011，一个有符号二进制数的补码形式。
<<	二进制左移运算符。将一个运算对象的各二进制位全部左移若干位（左边的二进制位丢弃，右边补0）。	A << 2 将得到 240，即为 1111 0000
>>	二进制右移运算符。将一个数的各二进制位全部右移若干位， 正数左补0，负数左补1 ，右边丢弃。	A >> 2 将得到 15，即为 0000 1111



位运算符- 案例

```
/*ops_bits_examples*/
#include <stdio.h>
int main() {
    unsigned int a = 60; /* 60 = 0011 1100 */
    unsigned int b = 13; /* 13 = 0000 1101 */
    int c = 0;
    c = a & b; /* 12 = 0000 1100 */
    printf("Line 1 - c 的值是 %d\n", c );
    c = a | b; /* 61 = 0011 1101 */
    printf("Line 2 - c 的值是 %d\n", c );
    c = a ^ b; /* 49 = 0011 0001 */
    printf("Line 3 - c 的值是 %d\n", c );
    c = ~a; /* -61 = 1100 0011 */
    printf("Line 4 - c 的值是 %d\n", c );
    c = a << 2; /* 240 = 1111 0000 */
    printf("Line 5 - c 的值是 %d\n", c );
    c = a >> 2; /* 15 = 0000 1111 */
    printf("Line 6 - c 的值是 %d\n", c );
}
```

当左侧的代码被编译和执行时，它会产生下列结果：

Line 1 - c 的值是 12
Line 2 - c 的值是 61
Line 3 - c 的值是 49
Line 4 - c 的值是 -61
Line 5 - c 的值是 240
Line 6 - c 的值是 15



自增、自减运算符

自增/自减运算符是使**对象**的值**增加/减少 1** 的一元运算符。

- 操作数 (expr)
 - 整型, 实数型, 指针的**可修改左值**表达式
 - 常见操作数: 变量
- 值类别
 - 非左值, 即不能 `&(++i)`
- 求值顺序
 - 后缀形式: 先求值, 再修改对象
 - 前缀形式: 先修改对象, 再求值
- 优先级 (高)
- 隐式转换 (无)
 - 例如: `char i; sizeof(++i)` 返回1; `sizeof(i)` 返回 4

它们能拥有后缀形式:

`expr ++`

`expr --`

还有前缀形式:

`++ expr`

`-- expr`



++A和A++的区别

```
#include <stdio.h>
int main() {
    int c;
    int a = 10;
    c = a++;
    printf("先赋值后运算: \n");
    printf("Line 1 - c 的值是 %d\n", c );
    printf("Line 2 - a 的值是 %d\n", a );

    a = 10;
    c = a--;
    printf("Line 3 - c 的值是 %d\n", c );
    printf("Line 4 - a 的值是 %d\n", a );
    printf("先运算后赋值: \n");

    a = 10;
    c = ++a;
    printf("Line 5 - c 的值是 %d\n", c );
    printf("Line 6 - a 的值是 %d\n", a );

    a = 10;
    c = --a;
    printf("Line 7 - c 的值是 %d\n", c );
    printf("Line 8 - a 的值是 %d\n", a );
}
```

Ops_inc_dec.c

例子演示了 a++ 与 ++a 的区别:

先赋值后运算:

Line 1 - c 的值是 10

Line 2 - a 的值是 11

Line 3 - c 的值是 10

Line 4 - a 的值是 9

先运算后赋值:

Line 5 - c 的值是 11

Line 6 - a 的值是 11

Line 7 - c 的值是 9

Line 8 - a 的值是 9



关系运算符

假设变量 A 的值为 10，变量 B 的值为 20，则：

运算符	描述	实例
==	检查两个操作数的值是否相等，如果相等则条件为真。	(A == B) 为假。
!=	检查两个操作数的值是否相等，如果不相等则条件为真。	(A != B) 为真。
>	检查左操作数的值是否大于右操作数的值，如果是则条件为真。	(A > B) 为假。
<	检查左操作数的值是否小于右操作数的值，如果是则条件为真。	(A < B) 为真。
>=	检查左操作数的值是否大于或等于右操作数的值，如果是则条件为真。	(A >= B) 为假。
<=	检查左操作数的值是否小于或等于右操作数的值，如果是则条件为真。	(A <= B) 为真。



关系运算符-例子

```
#include <stdio.h>
int main() {
    int a = 21;
    int b = 10;
    int c ;
    if( a == b ) printf("Line 1 - a 等于 b\n" );
    else printf("Line 1 - a 不等于 b\n" );
    if ( a < b ) printf("Line 2 - a 小于 b\n" );
    else printf("Line 2 - a 不小于 b\n" );
    if ( a > b ) printf("Line 3 - a 大于 b\n" );
    else printf("Line 3 - a 不大于 b\n" );
    /* 改变 a 和 b 的值 */
    a = 5;
    b = 20;
    if ( a <= b ) printf("Line 4 - a 小于或等于 b\n" );
    if ( b >= a ) printf("Line 5 - b 大于或等于 a\n" );
}
```

Ops-comparison.c

当左侧的代码被编译和执行时，
它会产生下列结果：

Line 1 - a 不等于 b

Line 2 - a 不小于 b

Line 3 - a 大于 b

Line 4 - a 小于或等于 b

Line 5 - b 大于或等于 a

逻辑运算符

假设变量 A 的值为 1，变量 B 的值为 0，则：

运算符	描述	实例
&&	称为逻辑与运算符。如果两个操作数都非零，则条件为真。	(A && B) 为假。
	称为逻辑或运算符。如果两个操作数中有任意一个非零，则条件为真。	(A B) 为真。
!	称为逻辑非运算符。用来逆转操作数的逻辑状态。如果条件为真则逻辑非运算符将使其为假。	!(A && B) 为真。



逻辑运算符-例子

Ops_logical.c

```
#include <stdio.h>
int main() {
    int a = 5;
    int b = 20;
    int c ;
    if ( a && b ) printf("Line 1 - 条件为真\n" );
    if ( a || b )  printf("Line 2 - 条件为真\n" );
    /* 改变 a 和 b 的值 */
    a = 0;
    b = 10;
    if ( a && b ) printf("Line 3 - 条件为真\n" );
    else printf("Line 3 - 条件为假\n" );
    if ( !(a && b) ) printf("Line 4 - 条件为真\n" );
}
```

在C语言当中，所有非 0 的数都会是真也就是 1，而 0 会是假也就是仍然是 0。当左侧的代码被编译和执行时，它会产生下列结果：

Line 1 - 条件为真
Line 2 - 条件为真
Line 3 - 条件为假
Line 4 - 条件为真



举例：交换两个数

问题：如何交换两个数？



举例：交换两个数

swap.c

```
#include <stdio.h>
int main() {
    double a,b,c;

    scanf("%lf,%lf",&a,&b);

    c=a;
    a=b;
    b=c;

    printf("a=%lf,b=%lf\n",a,b);

    return 0;
}
```



举例：交换两个数

问题：有无更好的办法？



举例：交换两个数

swap1.c

```
#include <stdio.h>
int main() {
    double a,b;

    scanf("%lf,%lf",&a,&b);

    a = a + b;
    b = a - b;
    a = a - b;

    printf("a=%lf,b=%lf\n",a,b);

    return 0;
}
```

变量	a	b
初始值	x	y
a=a+b	x+y	y
b=a-b	x+y	x
a=a-b	y	x



举例：交换两个数

swap2.c

```
#include <stdio.h>
int main() {
    int a,b,c;

    scanf("%d,%d",&a,&b);

    a = a ^ b;
    b = b ^ a;
    a = a ^ b;

    printf("a=%d,b=%d\n",a,b);

    return 0;
}
```

变量	a	b
初始值	x	y
a=a^b	x^y	y
b=a^b	x^y	x^(y^y)
a=a^b	y	x



运算优化

```
int I,J;  
I = 257 /8;  
J = 456 % 32
```




运算优化

```
int I,J;  
I = 257 /8;  
J = 456 % 32
```

```
int I,J;  
I = 257 >>3;  
J = 456 - (456 >> 4 << 4);
```



位运算应用-Mask

A = 1010 1010

B = 0000 1111 (Mask, 掩码)

A&B = 0000 1010 屏蔽(清零)高4位

A|B = 1010 1111 置位低4位

A^B = 1010 0101 高4位直通, 低4位求反



按位与的实际用途

将一个数的某位清零

如果想将某个数的某个比特清零，只要另找一个数，让对应的该比特置0，然后使两数进行按位与&运算，即可达到将此数的特定比特清零的目的。

如：有数为00101011，想让该数从右数的第4比特变成0，其他位保持不变。我们另找一个数，该数从右数的第4比特设它为0，其他位全为1：11110111，将两个数进行&运算：

$$\begin{array}{r} 00101011 \\ \& 11110111 \\ \hline 00100011 \end{array}$$



按位与的实际用途

取一个数中某些指定位

如有一个两个字节整数 $a = (16a1)_{16}$ ，想取出其中的低字节。我们构造一个数 b ，它的高字节全为0，低字节全为1，即 $b = (00ff)_{16} = (255)_{10}$ ，只需将 a 与 b 按位与即可：

$$\begin{aligned} a &= (16a1)_{16} = (0001011010100001)_2 \\ \& b &= (00ff)_{16} = (0000000011111111)_2 \\ \hline (00a1)_{16} &= (00000000010100001)_2 \end{aligned}$$



异或的实际用途

1. 使特定位翻转：假设有01111010，想使其低4位翻转，即1变为0，0变为1。可以将它与00001111进行 $\wedge$ 运算，即

$$\begin{array}{r} 01111010 \\ \wedge 00001111 \\ \hline 01110101 \end{array}$$

要使哪几位翻转就将其进行 $\wedge$ 运算的该几位置为1即可。

2. 与0相 $\wedge$ ，保留原值。如 $012 \wedge 00 = 012$

$$\begin{array}{r} 00001010 \\ \wedge 00000000 \\ \hline 00001010 \end{array}$$

因为原数中的1与0进行 $\wedge$ 运算得1， $0 \wedge 0$ 得0，故保留原数。



取反的实际用途

若一个16位整数a，想使最低一位为0，可以将a与二进制数1111111111111110 (0177776)₈进行按位与：

```
      0 0 0 0 0 0 0 0 0 0 1 1 1 1 0 1
& 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0
-----
      0 0 0 0 0 0 0 0 0 0 1 1 1 1 0 0
```

但若将此程序移植到32位计算机上，由于整数用4个字节表示，想将最后一位变成0就不能用a&0177776了，应改用a & 037777777776。这个算法的移植性很差，可以改用

a=a&~1:

因为在以2个字节存一个整数时，1的二进制形式为0000000000000001，~1是1111111111111110。以4个字节存储一个整数时，~1是11111111111111111111111111111110。



赋值运算符

运算符	描述	实例
=	简单的赋值运算符，把右边操作数的值赋给左边操作数	$C = A + B$ 将把 $A + B$ 的值赋给 C
+=	加且赋值运算符，把右边操作数加上左边操作数的结果赋值给左边操作数	$C += A$ 相当于 $C = C + A$
-=	减且赋值运算符，把左边操作数减去右边操作数的结果赋值给左边操作数	$C -= A$ 相当于 $C = C - A$
*=	乘且赋值运算符，把右边操作数乘以左边操作数的结果赋值给左边操作数	$C *= A$ 相当于 $C = C * A$
/=	除且赋值运算符，把左边操作数除以右边操作数的结果赋值给左边操作数	$C /= A$ 相当于 $C = C / A$
%=	求模且赋值运算符，求两个操作数的模赋值给左边操作数	$C \% = A$ 相当于 $C = C \% A$
<<=	左移且赋值运算符	$C << = 2$ 等同于 $C = C << 2$
>>=	右移且赋值运算符	$C >> = 2$ 等同于 $C = C >> 2$
&=	按位与且赋值运算符	$C \& = 2$ 等同于 $C = C \& 2$
^=	按位异或且赋值运算符	$C \wedge = 2$ 等同于 $C = C \wedge 2$
=	按位或且赋值运算符	$C = 2$ 等同于 $C = C 2$



赋值运算符-例子

```
#include <stdio.h>
int main() {
    int a = 21;
    int c ;
    c = a;
    printf("Line 1 - = 运算符实例, c 的值 = %d\n", c );
    c += a;
    printf("Line 2 - += 运算符实例, c 的值 = %d\n", c );
    c -= a;
    printf("Line 3 - -= 运算符实例, c 的值 = %d\n", c );
    c *= a;
    printf("Line 4 - *= 运算符实例, c 的值 = %d\n", c );
    c /= a;
    printf("Line 5 - /= 运算符实例, c 的值 = %d\n", c );
    c = 200;
    c %= a;
    printf("Line 6 - %= 运算符实例, c 的值 = %d\n", c );
    c <<= 2;
    printf("Line 7 - <<= 运算符实例, c 的值 = %d\n", c );
    c >>= 2;
    printf("Line 8 - >>= 运算符实例, c 的值 = %d\n", c );
    c &= 2;
    printf("Line 9 - &= 运算符实例, c 的值 = %d\n", c );
    c ^= 2;
    printf("Line 10 - ^= 运算符实例, c 的值 = %d\n", c );
    c |= 2;
    printf("Line 11 - |= 运算符实例, c 的值 = %d\n", c );
}
```

ops_and_assign.c



赋值运算符-例子

```
#include <stdio.h>
int main() {
    int a = 21;
    int c ;
    c = a;
    printf("Line 1 - = 运算符实例, c 的值 = %d\n", c );
    c += a;
    printf("Line 2 - += 运算符实例, c 的值 = %d\n", c );
    c -= a;
    printf("Line 3 - -= 运算符实例, c 的值 = %d\n", c );
    c *= a;
    printf("Line 4 - *= 运算符实例, c 的值 = %d\n", c );
    c /= a;
    printf("Line 5 - /= 运算符实例, c 的值 = %d\n", c );
    c = 200;
    c %= a;
    printf("Line 6 - %= 运算符实例, c 的值 = %d\n", c );
    c <<= 2;
    printf("Line 7 - <<= 运算符实例, c 的值 = %d\n", c );
    c >>= 2;
    printf("Line 8 - >>= 运算符实例, c 的值 = %d\n", c );
    c &= 2;
    printf("Line 9 - &= 运算符实例, c 的值 = %d\n", c );
    c ^= 2;
    printf("Line 10 - ^= 运算符实例, c 的值 = %d\n", c );
    c |= 2;
    printf("Line 11 - |= 运算符实例, c 的值 = %d\n", c );
}
```

当左侧的代码被编译和执行时，它会产生下列结果：

Line 1 - = 运算符实例, c 的值 = 21

Line 2 - += 运算符实例, c 的值 = 42

Line 3 - -= 运算符实例, c 的值 = 21

Line 4 - *= 运算符实例, c 的值 = 441

Line 5 - /= 运算符实例, c 的值 = 21

Line 6 - %= 运算符实例, c 的值 = 11

Line 7 - <<= 运算符实例, c 的值 = 44

Line 8 - >>= 运算符实例, c 的值 = 11

Line 9 - &= 运算符实例, c 的值 = 2

Line 10 - ^= 运算符实例, c 的值 = 0

Line 11 - |= 运算符实例, c 的值 = 2



其他运算符

运算符	描述	实例
sizeof()	返回变量的大小。	sizeof(a) 将返回 4, 其中 a 是整数。
&	返回变量的地址。	&a; 将给出变量的实际地址。
*	指向一个变量。	*a; 将指向一个变量。
? :	条件表达式	如果条件为真 ? 则值为 X : 否则值为 Y



其他运算符-例子

```
#include <stdio.h>
int main() {
    int a = 4;
    short b;
    double c;
    int* ptr;
    /* sizeof 运算符实例 */
    printf("Line 1 - 变量 a 的大小 = %lu\n", sizeof(a) );
    printf("Line 2 - 变量 b 的大小 = %lu\n", sizeof(b) );
    printf("Line 3 - 变量 c 的大小 = %lu\n", sizeof(c) );
    /* & 和 * 运算符实例 */
    ptr = &a; /* 'ptr' 现在包含 'a' 的地址 */
    printf("a 的值是 %d\n", a);
    printf("*ptr 是 %d\n", *ptr);
    /* 三元运算符实例 */
    a = 10;
    b = (a == 1) ? 20 : 30;
    printf("b 的值是 %d\n", b );
    b = (a == 10) ? 20 : 30;
    printf("b 的值是 %d\n", b );
}
```

ops_others.c

当左侧的代码被编译和执行时，它会产生下列结果：

Line 1 - 变量 a 的大小 = 4

Line 2 - 变量 b 的大小 = 2

Line 3 - 变量 c 的大小 = 8

a 的值是 4

*ptr 是 4

b 的值是 30

b 的值是 20



运算符优先级

运算符的优先级确定表达式中项的组合。这会影响到一个表达式如何计算。某些运算符比其他运算符有更高的优先级，例如，乘除运算符具有比加减运算符更高的优先级。

例如 $x = 7 + 3 * 2$ ，在这里， x 被赋值为 13，而不是 20，因为运算符 $*$ 具有比 $+$ 更高的优先级，所以首先计算乘法 $3*2$ ，然后再加上 7。

下表将按运算符优先级从高到低列出各个运算符，具有较高优先级的运算符出现在表格的上面，具有较低优先级的运算符出现在表格的下面。在表达式中，较高优先级的运算符会优先被计算。



类别	运算符	结合性
后缀	() [] -> . ++ --	从左到右
一元	+ - ! ~ ++ -- (type)* & sizeof	从右到左
乘除	* / %	从左到右
加减	+ -	从左到右
移位	<< >>	从左到右
关系	< <= > >=	从左到右
相等	== !=	从左到右
位与 AND	&	从左到右
位异或 XOR	^	从左到右
位或 OR		从左到右
逻辑与 AND	&&	从左到右
逻辑或 OR		从左到右
条件	?:	从右到左
赋值	= += -= *= /= %> >= <<= &= ^= =	从右到左
逗号	,	从左到右



运算符优先级-例子

```
#include <stdio.h>
int main() {
    int a = 20;
    int b = 10;
    int c = 15;
    int d = 5;
    int e;
    e = (a + b) * c / d; // ( 30 * 15 ) / 5
    printf("(a + b) * c / d 的值是 %d\n", e );
    e = ((a + b) * c) / d; // (30 * 15 ) / 5
    printf("((a + b) * c) / d 的值是 %d\n", e );
    e = (a + b) * (c / d); // (30) * (15/5)
    printf("(a + b) * (c / d) 的值是 %d\n", e );
    e = a + (b * c) / d; // 20 + (150/5)
    printf("a + (b * c) / d 的值是 %d\n", e );
    return 0;
}
```

ops_precedence_parenthesis.c



运算符优先级-例子

```
#include <stdio.h>
int main() {
    int a = 20;
    int b = 10;
    int c = 15;
    int d = 5;
    int e;
    e = (a + b) * c / d; // ( 30 * 15 ) / 5
    printf("(a + b) * c / d 的值是 %d\n", e );
    e = ((a + b) * c) / d; // (30 * 15 ) / 5
    printf("((a + b) * c) / d 的值是 %d\n", e );
    e = (a + b) * (c / d); // (30) * (15/5)
    printf("(a + b) * (c / d) 的值是 %d\n", e );
    e = a + (b * c) / d; // 20 + (150/5)
    printf("a + (b * c) / d 的值是 %d\n", e );
    return 0;
}
```

当左侧的代码被编译和执行时，它会产生下列结果：

$(a + b) * c / d$ 的值是 90

$((a + b) * c) / d$ 的值是 90

$(a + b) * (c / d)$ 的值是 90

$a + (b * c) / d$ 的值是 50



中山大學
SUN YAT-SEN UNIVERSITY

谢谢

中山大学计算机学院



讲课人：万海